

---

# Training JEPA Models for Plan Refinement

JEPA Planning Project Team  
University of Maryland

Semester proposal, 17 September 2026

## Abstract

A visual world model can predict a promising push and still be wrong about whether refining that plan improves it. We study which simulator experience helps a JEPA model make these comparisons. At a fixed budget of new rollout controls, should we execute intermediate and final versions of a plan from the same state, or collect final plans from more states? Our local PushT experiments show that gradient planning helps on average, but some refinements reduce predicted error while worsening the executed outcome. Small adaptation trials do not yet establish a benefit from paired collection. We propose a controlled study of this tradeoff, with an early decision point before committing the semester to a method.

## 1 Background and research question

In PushT, a circular pusher moves a T-shaped block. We supply a starting image and an image of the desired scene. An encoder converts each image into a vector of learned features. A Joint-Embedding Predictive Architecture, or JEPA, learns to predict features of another observation. Unlike an image generator, it need not reconstruct every pixel. I-JEPA predicts image regions and V-JEPA predicts video features [11, 12]. An action-conditioned world model also receives proposed controls, so it can predict their consequences [2, 13].

A planner tries candidate controls inside this model. Gradient planning changes the controls to reduce predicted distance to the goal. The predictor’s weights stay fixed during this search. Training does something different. It changes those weights using recorded actions and observed outcomes. Our project concerns the experience used for that training.

**Research question.** At a fixed budget of new rollout controls, does training on intermediate and final plans improve closed-loop PushT success more than collecting final plans from twice as many starts?

The comparison is concrete. One condition executes a final 20-step plan from 64 starts. Another executes two 20-step plans from 32 starts. Both spend 1,280 new steps. The paired condition sees how a refinement changes the outcome, but adds alternative-action experience at only half as many starts. Both conditions retain the same reference trajectories. We begin with the same released LeWorldModel (LeWM), freeze its image encoder, and adapt its action predictor [1]. We test whether any gain comes with better prediction of the sign and size of executed refinement changes, rather than lower average feature error. Prefix replay and reset costs are reported separately.

**Table 1. Related work.** These papers motivate the question. They do not establish that paired collection will win.

| Paper                | Relevant result or method                                                                                                     | What we must distinguish                                                                                                                 |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Train-test gap [4]   | Online World Modeling executes planner-generated trajectories and adapts the predictor on their outcomes.                     | Our final-plan condition follows this principle. Our question is how to allocate the same execution budget.                              |
| Metric alignment [6] | With the cross-entropy method (CEM), a sampling planner, ranking agreement drops sharply among the planner’s best candidates. | A good global prediction score may miss the comparisons used during refinement. Its reported planner is CEM, not our gradient optimizer. |
| CoCo [7]             | Same-state, multiple-action data support tests of action dependence.                                                          | Branching itself is established. We isolate intermediate versus final plans and the cost of fewer starts.                                |

**Practical aim.** Improve the decisions of an existing predictor without adding deployment-time networks or checks. This could matter when a robotics lab can collect additional simulator experience but must keep its planner fast. Exact replay is a simulator capability. We do not assume the same collection procedure is cheap on physical robots. The narrow comparison is a candidate contribution, not an established novelty claim.

## 2 Model and measurement

Let  $f_\phi(o)$  encode image  $o$  as  $z \in \mathbb{R}^{192}$ . The released model takes three recent feature vectors and the actions between them. One model step spans five primitive controls. Each control has two coordinates, so an action block  $u$  has ten. Its predictor  $g_\theta$  estimates the next feature vector [1]. With the encoder frozen, adaptation minimizes

$$\mathcal{L}(\theta) = \frac{1}{nd} \sum_{i=1}^n \left\| g_\theta(z_{0:2}^{(i)}, u_{0:2}^{(i)}) - z_3^{(i)} \right\|_2^2, \quad d = 192. \quad (1)$$

The index  $i$  selects a recorded trajectory window and  $n$  is the batch size. Within each window,  $0 : 2$  denotes three consecutive model steps. The target  $z_3^{(i)}$  comes from the next image we actually observed. Squaring and averaging the coordinate errors gives a next-step prediction loss. During training the history uses recorded features, a procedure called teacher forcing. During planning future features instead come from the model’s own predictions.

**The action gradient.** Stack four future blocks into  $A \in \mathbb{R}^{40}$ . Repeated prediction gives endpoint  $\hat{z}_H(A)$ , with  $H = 4$ . Set  $z_g = f_\phi(o_g)$  for the goal image and  $r = \hat{z}_H(A) - z_g$ . The planner minimizes  $\hat{C}(A) = r^\top r / d$ . A small control change  $\delta A$  gives

$$\delta r \approx J_A \delta A, \quad \delta \hat{C} \approx \frac{2}{d} r^\top J_A \delta A, \quad \nabla_A \hat{C} = \frac{2}{d} J_A^\top r, \quad J_A = \frac{\partial \hat{z}_H}{\partial A} \in \mathbb{R}^{192 \times 40}. \quad (2)$$

Column  $k$  of the Jacobian  $J_A$  describes how changing control coordinate  $k$  changes every predicted feature. Multiplying by the current error  $r$  tells us whether that change moves the prediction toward the goal. A basic update is  $A' = \Pi(A - \eta \nabla_A \hat{C})$ , with step size  $\eta$  and clipping  $\Pi$  to valid controls. We use Adam, which rescales updates using past gradients [27, 4]. This derivative describes the learned model. It is not a derivative of the actual simulator.

**Three distinct costs.** Execute the complete plan in the simulator, yielding image  $o_H(A)$  and three block landmarks  $p_k(A)$ . We measure

$$C(A) = \frac{\|f_\phi(o_H(A)) - z_g\|_2^2}{d}, \quad D(A) = \sqrt{\frac{1}{3} \sum_{k=1}^3 \|p_k(A) - p_k^g\|_2^2}. \quad (3)$$

$\hat{C}$  is predicted feature cost.  $C$  is feature cost after execution.  $D$  is block error in world pixels. Comparing  $\hat{C}$  with  $C$  tests prediction. Comparing  $C$  with  $D$  also tests whether the chosen feature distance matches the physical objective. Our block-only measure ignores the pusher. It is not the official PushT full-state success criterion [6, 37].

For a refinement from  $A$  to  $B$ , define  $\Delta C = C(B) - C(A)$  and  $\Delta \hat{C} = \hat{C}(B) - \hat{C}(A)$ . Negative means improvement. If  $e(A) = \hat{C}(A) - C(A)$ , subtraction gives

$$\Delta \hat{C} - \Delta C = e(B) - e(A). \quad (4)$$

This identity explains the measurement we need. A shared prediction bias can cancel when comparing two plans. A changing bias can reverse their order. If both cost errors are at most  $\epsilon$ , the comparison error is at most  $2\epsilon$ . Average prediction loss provides no such uniform guarantee. Paired outcomes measure this difference directly. They do not automatically teach it when training still uses only equation (1).

---

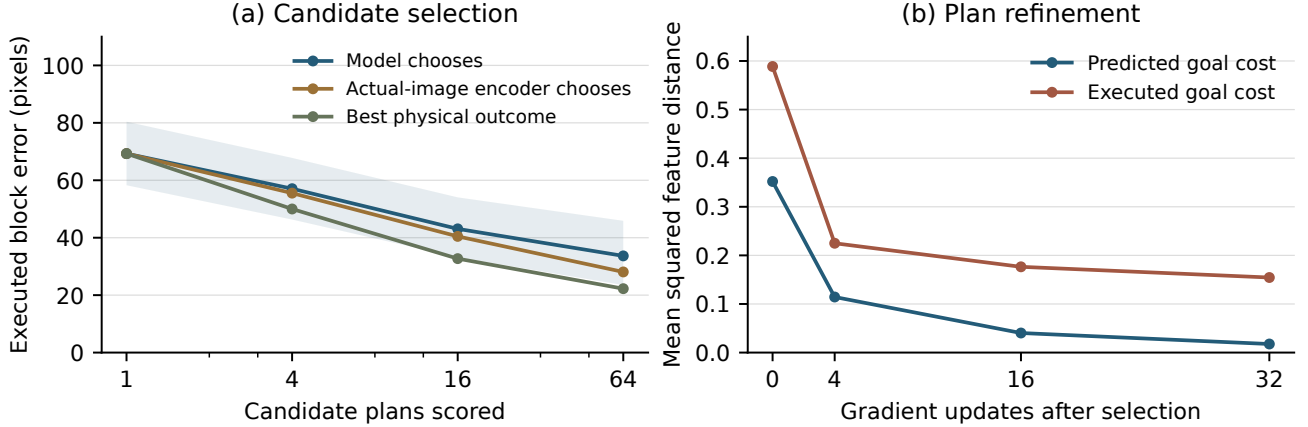
### Algorithm 1. Proposed matched-budget study

1. Save starting histories and feasible goals. Generate plans with one fixed collector model. Record intermediate plan  $A$  and refined plan  $B$ .
  2. Spend  $N$  new rollout controls either on  $B$  from  $N/20$  starts, or on  $(A, B)$  from  $N/40$  starts. Replay the identical prefix before each execution.
  3. Fit copied predictors using equation (1), identical old-data replay and matched updates. Freeze the encoder and all collection choices.
  4. Evaluate complete planners on sealed tasks. Measure executed goal error, success, latency and the signs of predicted versus executed refinements.
-

### 3 Local experiments

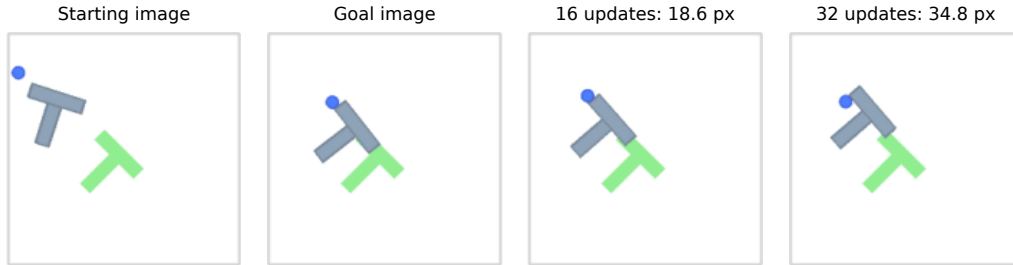
We ran the complete released LeWM checkpoint on an Apple M2 Pro CPU. Replay, reference-rollout and finite-difference checks passed. The accompanying experiment record gives tolerances, source versions and limitations.

After exploring 12 episodes, we repeated the diagnostic on 24 different episodes. We selected starts near a moving block and reachable goal images 20 controls ahead. We executed 64 plans per start, drawn from other expert trajectories, then refined the model’s best plan for 32 Adam updates. We replayed the same prefix before executing each refinement. This is an *open-loop* diagnostic, not a reproduction of LeWM’s closed-loop benchmark [1].



**Figure 1. Measured planning behavior on 24 tasks.** Left: horizontal position counts candidate plans. Vertical position is mean executed block error, lower is better. Blue is model selection. The other curves use executed outcomes and are hindsight controls unavailable to a deployed planner. Candidate order is averaged within each task. Shading bootstraps the 24 task means. Right: the optimizer reduces predicted feature cost faster than executed feature cost. Both curves use the same units.

**Results.** More candidates helped. Mean block error fell from 69.3 pixels with one candidate to 33.7 with 64, then to 12.2 after gradient refinement. Between updates 16 and 32, predicted cost improved in all 24 tasks, but executed feature cost worsened in eight. Block error worsened in ten, with only two regressions exceeding five pixels. Mean performance still improved.



**Figure 2. Largest late physical regression, selected for inspection.** Grey is the movable block and blue is the pusher. The target is the grey block in the second image. The green T is fixed renderer scenery. The model’s predicted cost fell from 0.031 to 0.016 between the last two images, while executed block error rose from 18.6 to 34.8 pixels. This case is not typical performance.

**Adaptation pilot.** On 64 additional source episodes, we compared state coverage with same-state branching. Each condition used 1,280 new steps, the same 1,280-step reference buffer, 200 predictor updates and three adaptation seeds. The encoder stayed frozen. Table 2 evaluates the full planner, not just prediction loss. These follow-ups reuse the 24 diagnostic tasks as validation. They are exploratory, not independent confirmation.

**Table 2. Executed block error after 32 gradient updates.** Lower is better. Each entry averages the same 24 validation tasks. Seeds vary adaptation, not pretraining. Untouched checkpoint: 12.21 pixels.

| Training data                     | Mean  | Three adaptation seeds |
|-----------------------------------|-------|------------------------|
| 64 starts, one sampled plan       | 11.12 | 9.51, 10.79, 13.06     |
| 16 starts, four sampled plans     | 12.92 | 12.75, 12.66, 13.35    |
| 64 starts, final plan             | 12.02 | 12.00, 11.87, 12.20    |
| 32 starts, intermediate and final | 11.89 | 11.17, 13.00, 11.49    |

Neither comparison establishes a benefit from branching.

## 4 Semester plan

**Limits.** We used one pretrained checkpoint, selected contact-near tasks and short plans. Public pretraining overlap is unknown. The pilot motivates measuring executed refinements and testing allocation. It does not yet show a scalable training improvement or justify a new regularizer.

**Main experiment.** Compare final-only collection, paired intermediate/final collection, and two independently initialized final plans per state. The third condition separates repeated starts from a relationship between refinements. Use  $N \in \{1,280, 5,120, 20,480\}$  additional controls and three adaptation seeds, giving 27 runs. Count reference data, prefix replay, planning compute and resets separately. Collection uses a common frozen model. Match training updates and tune optimizer settings once on validation data. This is a data-efficiency study, not a claim of a universal scaling law [8].

**Evaluation.** Seal 100 new start-goal tasks before the main runs. Primary evaluation uses the released closed-loop PushT success rule, which includes pusher position, block position and orientation [37]. Also report block error, executed feature cost, refinement disagreement and latency. Use paired task comparisons and uncertainty across adaptation seeds [31]. These seeds do not measure pretraining variance. Confirm the decisive result with independently trained initial models before claiming robustness. Add a fixed-budget sampling planner as a control [26, 25].

At week 4, continue paired collection only if its benefit survives matched final-plan controls and is large enough to matter in executed behavior. If pairs improve prediction but not decisions, test one explicit difference loss based on equation (4), using the same data and an equal-compute control [28, 5]. If sampling works but gradient refinement fails, examine local action responses before changing representation. If neither contrast persists, report the limit and stop this method. A null pilot alone is not a contribution. Architecture changes, text conditioning and large-scale pretraining stay outside this proposal [3, 9, 10].

**Table 3. Work by week.** Each month combines reading and implementation, with roughly half our effort on each. Four people rotate data, model, planning and evaluation work and review one another’s checks.

| Week | Read and understand                    | Implement or decide                                           |
|------|----------------------------------------|---------------------------------------------------------------|
| 1    | I-JEPA, LeWM [11, 1]                   | Reproduce loading, replay and basic prediction.               |
| 2    | DINO-WM, V-JEPA 2 [2, 13]              | Trace features, action blocks and image history.              |
| 3    | UPN, train-test gap [27, 4]            | Derive gradients and reproduce a closed-loop baseline.        |
| 4    | CoCo, exploration [7, 8]               | Run three collection controls and decide whether to continue. |
| 5    | Metric alignment, ActSWM [6, 5]        | Test executed refinement differences and controls.            |
| 6    | Evaluation, Sobolev training [31, 28]  | Freeze the justified comparison and seal evaluation tasks.    |
| 7    | LeJEPA, VICReg [14, 15]                | Audit frozen features and cache training data.                |
| 8    | JEPA design study [3]                  | Complete smallest-budget runs across seeds.                   |
| 9    | Fast LeWM, GRASP [9, 10]               | Complete larger budgets with fixed machinery.                 |
| 10   | DAGger, Plan2Explore [29, 30]          | Compare final-plan collection and state coverage.             |
| 11   | iCEM, PETS [26, 25]                    | Compare gradient and sampling planners fairly.                |
| 12   | Closest paper to the observed failure  | Run one justified follow-up, not multiple additions.          |
| 13   | Physical grounding, OGBench [34, 33]   | Test longer goals or wider action coverage, one only.         |
| 14   | Statistical Precipice [31]             | Confirm with new initial models and sealed tasks.             |
| 15   | Revisit mechanism and prior work       | Write effect sizes, failures and resource costs.              |
| 16   | Cross-review implementation and claims | Release the study and present what held up.                   |

**Compute.** The local adaptation pilots used cached features and took about 20–25 CPU seconds per fit plus evaluation. This does not predict full closed-loop benchmark cost. We budget a ceiling of **120 GPU-hours**: 12 for profiling, 27 for adaptations, 24 for evaluation, 12 for controls, 15 for initial-model replication and 30 contingency. These are scheduling allowances, not measured requirements. We request **two university GPUs with at least 24 GB each for 48 hours** (96 device-hours), plus 16 CPU cores, 64 GB RAM and 200 GB storage. Available Sparks, RTX 4090s and Teslas can cover the balance after checking their exact hardware. We will revise the request after profiling and reduce the grid before spending beyond it.

## Bibliography

**Supporting literature.** BYOL and SimSiam explain learning by matching views [16, 17]. DINOv2 supplies DINO-WM’s visual features, whereas MAE reconstructs pixels [18, 19]. PlaNet plans through latent dynamics. Dreamer trains a policy with imagined experience. TD-MPC2 and MuZero learn predictions tailored to control [20, 21, 22, 23]. MBPO motivates checking rollout error [24]. Energy-based models give a language for compatibility costs [35]. Video pretraining also motivates industrial world-model work [36]. We treat that company report as context rather than benchmark evidence. Diffusion Policy learns action sequences directly, unlike our explicit search through a world model [32]. These contrasts help us understand the fixed baseline. They are not extra components in our method.

Thirty-five research papers and two implementation or industry sources. Years follow first arXiv submission. Links point to current records. Recent preprints require scrutiny, and their conclusions are limited to their reported settings.

- [1] Maes et al. [LeWorldModel: Stable End-to-End Joint-Embedding Predictive Architecture from Pixels](#). 2026.
- [2] Zhou et al. [DINO-WM: World Models on Pre-trained Visual Features enable Zero-shot Planning](#). 2024.
- [3] Terver et al. [What Drives Success in Physical Planning with Joint-Embedding Predictive World Models?](#). 2025.
- [4] Parthasarathy et al. [Closing the Train-Test Gap in World Models for Gradient-Based Planning](#). 2025.
- [5] Gan et al. [ActSWM: Action-Sensitive World Models for Long-Horizon Planning in Open-World Games](#). 2026.
- [6] Wang et al. [Decision-Metric Alignment in Latent World Models: Diagnostics and Action-Conditioned Objectives for MPC Planning](#). 2026.
- [7] Shi et al. [Overcoming Statistical Bias in Action-Controllable World Models](#). 2026.
- [8] Wagenmaker et al. [Optimal Exploration for Model-Based RL in Nonlinear Systems](#). 2023.
- [9] Gao and Xu. [Fast LeWorldModel](#). 2026.
- [10] Psenka et al. [Parallel Stochastic Gradient-Based Planning for World Models](#). 2026.
- [11] Assran et al. [Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture](#). 2023.
- [12] Bardes et al. [Revisiting Feature Prediction for Learning Visual Representations from Video](#). 2024.
- [13] Assran et al. [V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning](#). 2025.
- [14] Balestriero and LeCun. [LeJEPA: Provable and Scalable Self-Supervised Learning Without the Heuristics](#). 2025.
- [15] Bardes et al. [VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning](#). 2021.
- [16] Grill et al. [Bootstrap your own latent: A new approach to self-supervised Learning](#). 2020.
- [17] Chen and He. [Exploring Simple Siamese Representation Learning](#). 2020.
- [18] Oquab et al. [DINOv2: Learning Robust Visual Features without Supervision](#). 2023.
- [19] He et al. [Masked Autoencoders Are Scalable Vision Learners](#). 2021.
- [20] Hafner et al. [Learning Latent Dynamics for Planning from Pixels](#). 2018.
- [21] Hafner et al. [Mastering Diverse Domains through World Models](#). 2023.
- [22] Hansen et al. [TD-MPC2: Scalable, Robust World Models for Continuous Control](#). 2023.
- [23] Schrittwieser et al. [Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model](#). 2019.
- [24] Janner et al. [When to Trust Your Model: Model-Based Policy Optimization](#). 2019.
- [25] Chua et al. [Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models](#). 2018.
- [26] Pinneri et al. [Sample-efficient Cross-Entropy Method for Real-time Planning](#). 2020.
- [27] Srinivas et al. [Universal Planning Networks](#). 2018.
- [28] Czarnecki et al. [Sobolev Training for Neural Networks](#). 2017.
- [29] Ross et al. [A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning](#). 2010.
- [30] Sekar et al. [Planning to Explore via Self-Supervised World Models](#). 2020.
- [31] Agarwal et al. [Deep Reinforcement Learning at the Edge of the Statistical Precipice](#). 2021.
- [32] Chi et al. [Diffusion Policy: Visuomotor Policy Learning via Action Diffusion](#). 2023.
- [33] Park et al. [OGBench: Benchmarking Offline Goal-Conditioned RL](#). 2024.
- [34] Liu et al. [Toward Physically Grounded JEPA World Models for Goal-Conditioned Robotic Planning](#). 2026.
- [35] Dawid and LeCun. [Introduction to Latent Variable Energy-Based Models: A Path Towards Autonomous Machine Intelligence](#). 2023.
- [36] D. Li and J. Li. [Scaling Video Pretraining with Imagination Models](#). Induction Labs technical post, July 2026. Company-reported evidence.
- [37] Galilai Group. [stable-worldmodel](#). PushT environment and LeWM rollout implementation. Commit 4821c8e. Accessed September 2026.